Track 3: QAOA and Optimisation

MaxCut, routing, and an honest look at approximation quality

Qiskit Fall Fest Hackathon · Oct 9, 2026 · IISER Bhopal (EECS Club and Physics Club)

Read this first. You have one week, a laptop, and AI assistants. The ten problem statements are suggestions, not a menu you must order from. You may pick one, extend one, combine several, or invent something more ambitious. Original, advanced and cross-track projects are welcome and will be judged on the same criteria.

What we reward. A project that works, that you can explain line by line, and that is honestly compared against a classical alternative. Using AI to write code is allowed and encouraged; being unable to explain or verify that code is not.

How to read the problem statements

Every problem has the same parts: a short background, the problem itself, the core tasks (what a complete submission does), stretch goals (what makes it excellent), and a hint on tools. Treat the core tasks as the minimum, not the ceiling.

Simulators first, hardware as a bonus. Everything here can be done on a simulator (Qiskit Aer or the statevector tools). Running a small version on real IBM hardware, with a clear comparison against the simulator, earns extra credit in any track.

This track contains: **Problem 5, Problem 6**.

Problem 5: MaxCut with QAOA, and how good is good enough

Background

MaxCut asks you to split the dots of a network into two groups so that as many links as possible run between the groups. It models problems such as clustering, circuit layout and scheduling, and it is the standard test problem for QAOA.

Problem

Solve MaxCut on a family of graphs with QAOA and measure how the quality of the answer depends on the number of layers, the graph and the optimiser.

Core tasks (the minimum for a complete submission)

1. Build the cost operator for a graph and the QAOA circuit with p layers. Do it yourself rather than only calling a ready-made solver.
2. Solve at least three graph types (for example a ring, a random graph and a 3-regular graph) with 6 to 12 nodes, for p = 1, 2 and 3.
3. Report the approximation ratio (the cut you found divided by the best possible cut, from brute force) and the probability of sampling the optimal cut.
4. Compare with a classical method: a greedy heuristic and the Goemans-Williamson approach, or at least a simple local search.

Stretch goals (what makes it excellent)

- Study the optimiser: compare COBYLA, SPSA and gradient-based methods, and test smart initial angles against random starts.
- Explore warm-starting QAOA from a classical solution, or adding a different mixer.
- Add a noise model and find the largest p that still helps.
- Run one small graph on real hardware.

Hints on tools

Qiskit Optimization turns a graph problem into a cost Hamiltonian; the Estimator and Sampler primitives run the loop. Always check your result against brute force on small graphs before trusting it on larger ones.

Problem 6: A routing problem: deliveries on a small map

Background

Routing questions, such as the best order to visit a set of places or how to split deliveries between two vehicles, are central to logistics. They are hard because the number of possible routes explodes as places are added. Encoding them for a quantum computer is itself a skill: rules such as "visit every place exactly once" become penalty terms in the cost function.

Problem

Encode a small routing problem as a QUBO or Ising problem, solve it with QAOA, and find where the approach breaks down.

Core tasks (the minimum for a complete submission)

1. Choose a travelling-salesperson problem with 4 or 5 locations, or a vehicle-routing problem with a few locations and two vehicles. State the distance matrix.
2. Write down the encoding: which qubit means what, the distance term, and the penalty terms that force a valid route. Count the qubits you need.
3. Solve with QAOA and report: how often a valid route appears, how often the best valid route appears, and the cost of the best route found.
4. Compare against brute force and against a classical heuristic, and discuss the effect of the penalty weight.

Stretch goals (what makes it excellent)

- Use a more qubit-efficient encoding and compare the results.
- Study how the fraction of invalid routes changes with the penalty weight and the number of layers.
- Scale to the largest instance your simulator can handle and plot runtime against problem size.
- Replace the plain mixer with one that keeps solutions valid, and show the improvement.

Hints on tools

The qubit count grows quickly (roughly with the square of the number of locations), so keep instances small and focus on a clean encoding and an honest comparison. Do not claim a quantum speedup; show what you actually measured.

Go beyond: combine tracks or invent your own

The most impressive entries will not stop at one box. None of these is required, and you are free to propose something entirely different.

Ideas that involve Track 3

Idea	Tracks combined	What makes it interesting
Logical qubits inside an optimiser	Steane code and QAOA or VQE	Does error protection improve the answer on a noisy simulator, or does the overhead cancel the gain?
QAOA for code design	QAOA and error correction	Search for good parity-check matrices or qubit layouts as an optimisation problem.

Open invitation. If you have your own idea, such as a quantum game, an algorithm tutor, a benchmark of a quantum algorithm from our workshop (Grover, phase estimation, the QFT), or a tool that makes any of the above easier for the next student, propose it. Write three sentences on the problem, the method, and how you will know it worked, and check with the organisers early in the week.

How to scope a combined project. Get the smallest version of each part working separately first, then connect them. A small, honest, working combination beats an ambitious one that does not run.

What every team submits

Whichever problem you choose (or invent), your submission has the same five parts:

- A code repository or notebook that runs from a clean start, with versions of the packages listed.
- A write-up PDF (two to four pages) covering the problem, method, results and an honest limits section.
- At least one figure that shows the main result.
- A short AI-use note: which tools you used, one thing the AI got wrong, and how you caught it.
- A five-minute demo followed by questions; every team member must be able to take questions.

Bonus. Bonus points are available for a hardware run on a real IBM backend, with a clear comparison against the simulator.

Full submission rules are in *docs/guidelines_and_submission_rules.pdf*.

How you are judged

The same scoring applies to every problem and to original projects alike.

Criterion	Points	What judges look for
Correctness and verification	30	Results checked against an exact or classical reference, tests, sensible error bars
Understanding and insight	25	Clear explanation of why the method works or fails, shown in the write-up and in the live questions
Honest comparison and limits	20	A fair classical baseline, stated assumptions, no unsupported claims of quantum advantage
Ambition and originality	15	Stretch goals, combined tracks, or a new idea beyond the suggested statements
Clarity of demo and code	10	Readable, reproducible code and a clear five-minute story
Total	**100**	Bonus points are available on top for a real IBM hardware run (see Bonus).

> **On using AI.** AI assistants are welcome at every step. The AI-use note and the live questions exist because the person submitting is responsible for the result: you should be able to explain each circuit, formula and plot, and show how you verified it.